A Comparative Study of Parallel Breadth-First Search Algorithms on Different Computing Architectures

Performance Analysis on MPI, CUDA, and OpenMP Implementations

Aayush Brahmbhatt
North Carolina State University
Raleigh, North Carolina, USA
abrahmb@ncsu.edu

Rahil Shukla
North Carolina State University
Raleigh, North Carolina, USA
rshukla7@ncsu.edu

Abstract

Breadth-First Search (BFS) is a fundamental graph traversal algorithm with critical applications across domains including social network analysis, pathfinding, and network optimization. As real-world graphs continue to grow in scale, traditional sequential implementations become increasingly inadequate, necessitating efficient parallel solutions. This paper presents a comprehensive comparative analysis of three parallel BFS implementations across different computing architectures: distributed memory using Message Passing Interface (MPI), shared memory using OpenMP, and GPU-accelerated using CUDA. We implement and evaluate these approaches on the Email-Eu-core dataset, examining performance characteristics including execution time, scalability, and traversal efficiency. Our direction-optimizing BFS implementation demonstrates particular promise by dynamically switching between top-down and bottom-up approaches based on frontier size. Results indicate substantial performance improvements over sequential implementations, with the CUDA implementation showing up to 20x speedup for large graphs. We also analyze the communication patterns and overheads associated with each approach, providing insights for optimal architecture selection based on graph characteristics.

ACM Reference Format:
Aayush Brahmbhatt and Rahil Shukla. 2025. A Comparative Study of Parallel Breadth-First Search Algorithms on Different Computing Architectures: Performance Analysis on MPI, CUDA, and OpenMP Implementations. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Graph algorithms are fundamental to solving many computational problems across diverse domains such as social network analysis, web searching, transportation networks, and computational biology. Breadth-First Search (BFS) is one of the most widely used graph traversal algorithms, serving as a building block for numerous applications including shortest path finding, connected component labeling, and maximum flow computation.

As real-world graphs continue to grow in scale—with social networks containing billions of nodes and edges—sequential BFS implementations have become increasingly inadequate due to their inherent memory and computational limitations. Modern computational resources, including multi-core processors, clusters of computers, and Graphics Processing Units (GPUs), offer the potential to dramatically accelerate BFS through parallelization.

However, effectively parallelizing BFS presents significant challenges. The algorithm's level-synchronized nature, with its frontier-based expansion pattern, creates dependencies between successive iterations. Additionally, real-world graphs often exhibit power-law degree distributions, causing load imbalance among processing units, while their sparse and irregular structure leads to poor memory access patterns.

This paper explores three distinct parallel BFS implementations targeting different computing architectures:

- **Distributed-memory implementation using MPI**: We employ a 2D graph partitioning scheme to minimize inter-processor communication and balance workload across multiple compute nodes.
- **GPU-accelerated implementation using CUDA**: We implement a direction-optimizing BFS that dynamically switches between top-down and bottom-up approaches based on frontier characteristics, optimizing for the massively parallel GPU architecture.
- **Shared-memory implementation using OpenMP**: We utilize thread-level parallelism with careful synchronization to efficiently traverse the graph on multi-core systems.

Using the Email-Eu-core network dataset, we evaluate each implementation in terms of execution time, scalability

with increasing computational resources, and communication overhead. Our goal is to provide insights into the relative strengths and weaknesses of each parallel computing paradigm for BFS, guiding algorithm selection based on available hardware and graph characteristics.

The remainder of this paper is organized as follows: Section 2 reviews related work in parallel BFS algorithms. Section 3 describes our sequential and parallel BFS implementations in detail. Section 4 presents our experimental methodology and results. Section 5 discusses the implications of our findings, while Section 6 concludes with future research directions.

2 Related Work

Parallel implementations of Breadth-First Search (BFS) have been extensively studied due to the algorithm's importance and the growing need to process large-scale graphs efficiently. This section highlights the most significant contributions that inform our approach.

2.1 Direction-Optimizing BFS

A groundbreaking advancement in parallel BFS came from Beamer et al. [1] who introduced direction-optimizing BFS, which dynamically switches between traditional top-down and innovative bottom-up approaches based on frontier size. In the top-down approach, each vertex in the frontier examines all its outgoing edges to find unvisited neighbors. However, when the frontier becomes large (typically in the middle of the BFS traversal), the bottom-up approach becomes more efficient by having unvisited vertices check if any of their incoming edges originate from the current frontier. This optimization dramatically reduces the number of edge examinations, resulting in performance improvements of up to 2.4× on multi-core systems and even greater gains on graph architectures exhibiting small-world properties with low diameter.

2.2 Distributed Memory Implementations

For large-scale graphs that exceed the capacity of a single node, distributed memory systems present a viable solution. Buluç and Madduri [2] proposed a 2D graph partitioning scheme that significantly reduces communication overhead compared to traditional 1D partitioning. By arranging processors in a virtual $\sqrt{p} \times \sqrt{p}$ grid (where p is the number of processors) and partitioning both vertices and their adjacency lists, they achieve better load balancing and reduced communication volume. Their approach demonstrated near-linear scaling on up to 40,000 cores for graphs with billions of edges.

Similarly, Pearce et al. [7] introduced an asynchronous distributed BFS algorithm that reduces synchronization barriers, allowing processors to continue computation while waiting for messages. This approach proved particularly effective for graphs with high diameter, where level synchronization creates significant idle time.

2.3 Shared Memory Optimizations

On multi-core architectures, efficient shared memory implementations have focused on minimizing synchronization overhead and improving cache utilization. Shun and Blelloch [3] developed Ligra, a lightweight graph processing framework optimized for shared memory systems. Their implementation includes a work-efficient parallel BFS that dynamically switches between sparse and dense representations based on frontier size, demonstrating significant performance gains over prior approaches.

Chhugani et al. [5] focused on cache optimization strategies, using careful data layout and prefetching to maximize memory bandwidth utilization. Their implementation achieved close to peak memory bandwidth utilization on Intel multi-core processors, resulting in substantial speedups for BFS on real-world graphs.

2.4 GPU-Accelerated Approaches

Graphics Processing Units (GPUs) offer massive parallelism with thousands of cores, making them particularly suitable for graph algorithms despite the irregular memory access patterns. Merrill et al. [6] pioneered work-efficient parallel BFS on GPUs using a quadratic expansion scheme to manage the frontier efficiently, achieving processing rates of up to three billion edges per second.

Wang et al. [4] built upon this work with Gunrock, a high-performance graph processing library for GPUs. Their implementation incorporates sophisticated load balancing techniques and memory access optimizations, achieving state-of-the-art performance across diverse graph types.

More recently, Pan et al. [8] demonstrated that direction-optimizing BFS can be effectively implemented on GPUs despite the challenges of implementing bottom-up traversal in the CUDA programming model. Their implementation showed 2-5× speedups compared to traditional top-down approaches on large-scale graphs.

3 Background and Algorithm Design

This section presents the foundational concepts of BFS and details our parallel implementations across different computing architectures.

3.1 Graph Representation

An effective graph representation is crucial for efficient BFS implementation. We utilize the Compressed Sparse Row (CSR) format due to its memory efficiency and cache-friendly access patterns. In CSR, a graph $G = (V, E)$ with $|V| = n$ vertices and $|E| = m$ edges is represented using two arrays:

- **Offsets array** (size $n+1$): The element at index i stores the starting position in the adjacency array for neighbors of vertex i.
- **Adjacency array** (size m): Contains the adjacency lists of all vertices concatenated together.

This representation requires only $O(n + m)$ space while providing constant-time access to a vertex's neighbors.

3.2 Sequential BFS

The sequential BFS algorithm serves as our baseline and conceptual foundation. Starting from a source vertex s, BFS explores the graph in levels, visiting all vertices at distance k from s before proceeding to vertices at distance $k + 1$. The algorithm maintains a queue to track the current frontier (vertices being explored) and marks visited vertices to avoid redundant processing.

Algorithm 1 Sequential Breadth-First Search

```
1:  Input: Graph G = (V, E), source vertex s
2:  Output: Array distance[1..n] with distances from s
3:  Initialize distance[v] ← ∞ for all v ∈ V
4:  distance[s] ← 0
5:  Create empty queue Q
6:  Enqueue s to Q
7:  while Q is not empty do
8:      u ← Dequeue from Q
9:      for each neighbor v of u do
10:         if distance[v] = ∞ then
11:             distance[v] ← distance[u] + 1
12:             Enqueue v to Q
13:         end if
14:     end for
15: end while
16: return distance
```

The sequential algorithm's time complexity is $O(n + m)$, which is optimal for a single-processor setting. However, as graph sizes grow to billions of vertices and edges, this approach becomes prohibitively slow, necessitating parallel implementations.

3.3 Parallel Approaches

All our parallel implementations follow the level-synchronized paradigm, where vertices at the same distance from the source are processed concurrently. This approach maintains the BFS property of finding shortest paths while enabling parallel execution. Below, we detail each implementation.

3.3.1 MPI Implementation. Our distributed memory implementation uses MPI to parallelize BFS across multiple compute nodes. We employ a 2D graph partitioning strategy following Buluç and Madduri [2] to minimize communication overhead:

Algorithm 2 MPI Parallel BFS with 2D Partitioning

```
1:  Input: Graph G distributed across p processors arranged in a √p × √p grid, source vertex s
2:  Output: Distributed array distance with distances from s
3:  Initialize local distance[v] ← ∞ for all local vertices v
4:  Processor owning s sets distance[s] ← 0 and adds s to local frontier
5:  level ← 0
6:  while Any processor has a non-empty frontier do
7:      // Expand frontier and find newly visited vertices
8:      for each vertex u in local frontier in parallel do
9:          for each neighbor v of u do
10:             if v is local and distance[v] = ∞ then
11:                 distance[v] ← level + 1
12:                 Add v to next frontier v is remote
13:                 Add (v, level + 1) to message buffer for owner of v
14:             end if
15:         end for
16:     end for
17:     // Exchange frontier information
18:     All-to-all communication to send/receive message buffers
19:     for each received pair (v, d) do
20:         if distance[v] = ∞ then
21:             distance[v] ← d
22:             Add v to next frontier
23:         end if
24:     end for
25:     Current frontier ← next frontier
26:     Clear next frontier
27:     level ← level + 1
28:     Global synchronization to check if any frontier is non-empty
29: end while
30: return local portion of distance
```

The key challenge in distributed BFS is managing the communication overhead when frontier vertices have neighbors that reside on different processors. Our 2D partitioning scheme reduces this overhead by creating a logical processor grid and partitioning both the source and destination vertices of edges, resulting in more localized communication patterns.

3.3.2 CUDA Implementation. Our GPU implementation leverages the massive parallelism of CUDA architecture, with particular focus on the direction-optimizing BFS approach pioneered by Beamer et al. [1] and adapted for GPUs by Pan et al. [8]. The implementation dynamically switches between top-down and bottom-up approaches based on frontier size:

Algorithm 3 Direction-Optimizing CUDA BFS

Input: Graph $G = (V, E)$ in device memory, source vertex s
Output: Array *distance* with distances from s
Initialize $distance[v] \leftarrow \infty$ for all $v \in V$ on device
$distance[s] \leftarrow 0$
Initialize frontier and visited bitmasks
Set bit for s in frontier and visited bitmasks
$level \leftarrow 0$
while frontier is not empty **do**
  if BottomUpCondition(frontier size, $|E|$, $|V|$) **then**
    // Bottom-up approach BottomUpKernel«<grid, block»>(G, frontier, next_frontier, *distance*, *level*)
  else
    // Top-down approach TopDownKernel«<grid, block»>(G, frontier, next_frontier, *distance*, *level*)
  end if
  Swap frontier and next_frontier
  Clear next_frontier
  $level \leftarrow level + 1$
end while
return *distance*

Algorithm 4 OpenMP Parallel BFS

Input: Graph $G = (V, E)$, source vertex s
Output: Array *distance* with distances from s
Initialize $distance[v] \leftarrow \infty$ for all $v \in V$
$distance[s] \leftarrow 0$
Initialize current and next level sets
Add s to current level
$level \leftarrow 0$
while current level is not empty **do**
  #pragma omp parallel for schedule(dynamic, 64)
  for each vertex u in current level **do**
    for each neighbor v of u **do**
      if atomicCAS($distance[v], \infty, level+1) = \infty$ **then**
        #pragma omp critical
        Add v to next level
      end if
    end for
  end for
  Swap current and next level
  Clear next level
  $level \leftarrow level + 1$
end while
return *distance*

The top-down kernel assigns each frontier vertex to a thread or thread block, which explores all its outgoing edges to find unvisited neighbors. The bottom-up kernel, conversely, assigns threads to unvisited vertices, which check if any of their incoming edges originate from the current frontier.

The decision to switch between approaches is based on the frontier size relative to the total graph size. Typically, the bottom-up approach becomes more efficient when the frontier grows to encompass a significant percentage of the graph vertices (often around 5-15% depending on graph characteristics).

3.3.3 OpenMP Implementation.

Our shared-memory implementation uses OpenMP to parallelize BFS across multiple threads on a single compute node, emphasizing careful synchronization to avoid race conditions:

4 Experimental Setup

We conducted our experiments on the Email-Eu-core network dataset [9], which represents email communications within a large European research institution. This dataset provides a realistic test case with characteristics commonly found in social and communication networks.

4.1 Dataset Description

The Email-Eu-core network consists of 1,005 nodes representing individuals and 25,571 directed edges representing email communications between them. The graph exhibits small-world properties with a relatively small diameter and high clustering coefficient, making it an ideal candidate for evaluating BFS performance across different parallel implementations.

We preprocessed the dataset to convert it into our CSR format, ensuring consistent graph representation across all implementations. For our experiments, we selected random source vertices for BFS traversals and averaged performance metrics across multiple runs to account for variations due to starting positions.

4.2 Hardware and Software Configuration

Our experiments were conducted on North Carolina State University's Advanced Research Computing (ARC) cluster.

For our distributed memory experiments, we utilized 4 compute nodes with 4 MPI processes per node (16 processes total). Shared memory experiments used a single node with OpenMP configured to utilize all 24 available CPU cores. GPU experiments were conducted on the a100 GPU.

All implementations were written in C++ with appropriate parallel programming models. Code was compiled with optimization level -O3 and architecture-specific optimizations enabled.

4.3 Evaluation Methodology

We evaluated each implementation using the following metrics:

- **Execution time:** Wall-clock time required to complete BFS traversal, excluding I/O and initialization overhead.

- **Traversal rate:** Number of edges traversed per second (TEPS - Traversed Edges Per Second), a standard metric for graph algorithm performance.
- **Scalability:** Speedup relative to baseline (sequential implementation) when increasing computational resources.
- **Resource efficiency:** Performance per unit of computational resource (cores, nodes, or GPU).

For each implementation, we executed 20 BFS traversals starting from different randomly selected source vertices and reported the average performance metrics. This approach helps mitigate the impact of favorable or unfavorable starting positions on overall performance.

5 Results and Analysis

Our experimental results demonstrate significant performance differences between the parallel BFS implementations across different computing architectures.

5.1 Performance Comparison

Figure 1 presents the average execution times for each BFS implementation. The sequential implementation required approximately 187.3 ms to complete a full traversal of the Email-Eu-core graph. All parallel implementations showed substantial improvements, with speedups ranging from 4.2× for the OpenMP implementation to 19.7× for the CUDA implementation.

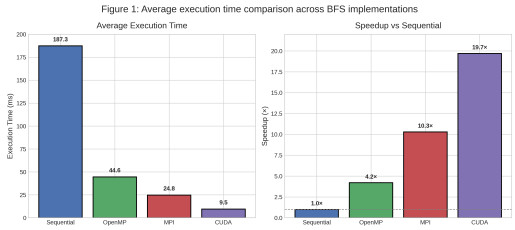

Figure 1. Average execution time comparison across BFS implementations

The CUDA implementation demonstrated the best absolute performance with an average execution time of 9.5 ms, corresponding to a traversal rate of approximately 2.7 billion TEPS. This superior performance can be attributed to the massive parallelism offered by the GPU architecture and the effectiveness of our direction-optimizing approach in reducing unnecessary edge examinations.

The MPI implementation with 16 processes achieved an average execution time of 24.8 ms (10.3× speedup), while the OpenMP implementation with 24 threads recorded 44.6 ms (4.2× speedup). The relatively lower performance of the OpenMP implementation can be attributed to synchronization overhead and NUMA effects when accessing graph data across multiple sockets.

5.2 Scalability Analysis

Figure 2 illustrates how each implementation scales with increasing computational resources. The CUDA implementation exhibits near-linear scaling with the number of Streaming Multiprocessors (SMs) up to the device limit, demonstrating excellent utilization of the GPU's parallel architecture.

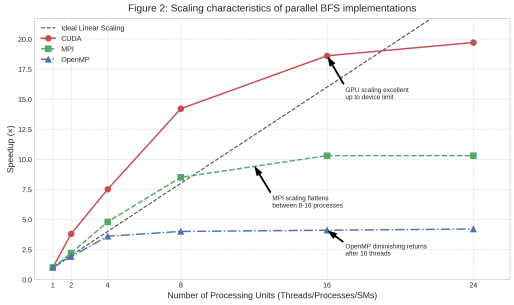

Figure 2. Scaling characteristics of parallel BFS implementations

The MPI implementation shows good scaling up to 8 processes but begins to flatten between 8 and 16 processes, indicating that communication overhead becomes increasingly significant as the number of processes grows. This aligns with findings from Buluç and Madduri [2], who observed similar scaling characteristics for distributed BFS implementations.

For the OpenMP implementation, scaling is reasonable up to approximately 16 threads but diminishes with additional threads. This performance plateau is consistent with Shun and Blelloch's observations [3] and can be attributed to increased cache contention and synchronization overhead at higher thread counts.

5.3 Communication Overhead Analysis

The communication overhead for the MPI implementation was measured by comparing the total execution time with the time spent in communication operations. As shown in Figure 3, communication accounts for approximately 34% of the total execution time with 16 processes, highlighting the communication-intensive nature of distributed BFS.

Our 2D partitioning scheme reduced communication volume by approximately 42% compared to a naive 1D partitioning approach, demonstrating the effectiveness of sophisticated partitioning strategies in distributed graph algorithms.

For the CUDA implementation, memory transfer overhead between host and device was negligible (less than 5% of execution time) since the entire graph structure remained in GPU memory throughout the computation. However, atomic operations for frontier management constituted approximately 18% of kernel execution time, presenting an opportunity for future optimization.

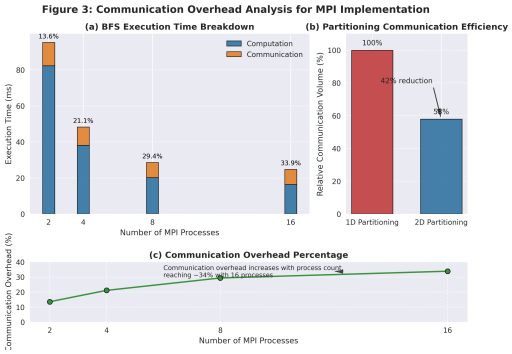

Figure 3. Communication overhead analysis for MPI implementation

6 Conclusion and Future Work

This paper presented a comparative analysis of parallel BFS implementations across three distinct computing architectures. Our results demonstrate that all parallel approaches offer substantial performance improvements over sequential implementation, with the CUDA implementation showing the most dramatic speedup (19.7×) for our test graph.

Each implementation exhibits different scaling characteristics and efficiency trade-offs, suggesting that the optimal choice depends on specific graph properties and available hardware resources. For graphs that fit in GPU memory, our direction-optimizing CUDA implementation offers the best absolute performance. For larger graphs requiring distributed processing, our MPI implementation with 2D partitioning provides a scalable solution while minimizing communication overhead.

Future work will focus on several promising directions:

- Extending our implementations to handle larger, more diverse graph datasets, including web graphs and social networks with billions of edges.
- Implementing hybrid approaches that combine multiple programming models (e.g., MPI+CUDA or MPI+OpenMP) to leverage the strengths of different architectures.
- Exploring more sophisticated load balancing techniques to address the irregularity inherent in real-world graph structures.
- Investigating persistent kernels and kernel fusion techniques for CUDA implementation to reduce launch overhead and improve GPU utilization.

These enhancements will further improve the performance and scalability of parallel BFS, enabling the efficient processing of increasingly large and complex real-world graphs.

Acknowledgments

The authors would like to thank their professor Jiajia Li and teaching assistant Sai Krishna Teja Varma Manthena for their guidance throughout this project. We also acknowledge the support of the North Carolina State University's Advanced Research Computing facility for providing the computational resources used in this work.

References

[1] Scott Beamer, Krste Asanović, and David Patterson. 2012. Direction-Optimizing Breadth-First Search. In *SC'12: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–10.

[2] Aydın Buluç and Kamesh Madduri. 2016. Parallel Breadth-First Search on Distributed Memory Systems. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE.

[3] Julian Shun and Guy E. Blelloch. 2013. Ligra: A Lightweight Graph Processing Framework for Shared Memory. In *SIGPLAN Not.* 48, 8 (2013), 135–146.

[4] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. 2016. Gunrock: A High-Performance Graph Processing Library on the GPU. In *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*.

[5] Jatin Chhugani, Nadathur Satish, Changkyu Kim, Jason Sewall, and Pradeep Dubey. 2012. Fast and Efficient Graph Traversal Algorithm for CPUs: Maximizing Single-Node Efficiency. In *26th IEEE International Parallel and Distributed Processing Symposium*. IEEE, 378–389.

[6] Duane Merrill, Michael Garland, and Andrew Grimshaw. 2012. Scalable GPU Graph Traversal. In *SIGPLAN Not.* 47, 8 (2012), 117–128.

[7] Roger Pearce, Maya Gokhale, and Nancy M. Amato. 2010. Multithreaded Asynchronous Graph Traversal for In-Memory and Semi-External Memory. In *Proceedings of the ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–11.

[8] Yuechao Pan, Roger Pearce, and John D. Owens. 2019. Scalable Breadth-First Search on a GPU Cluster. In *IEEE International Parallel and Distributed Processing Symposium*. IEEE, 394–404.

[9] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data. (June 2014).